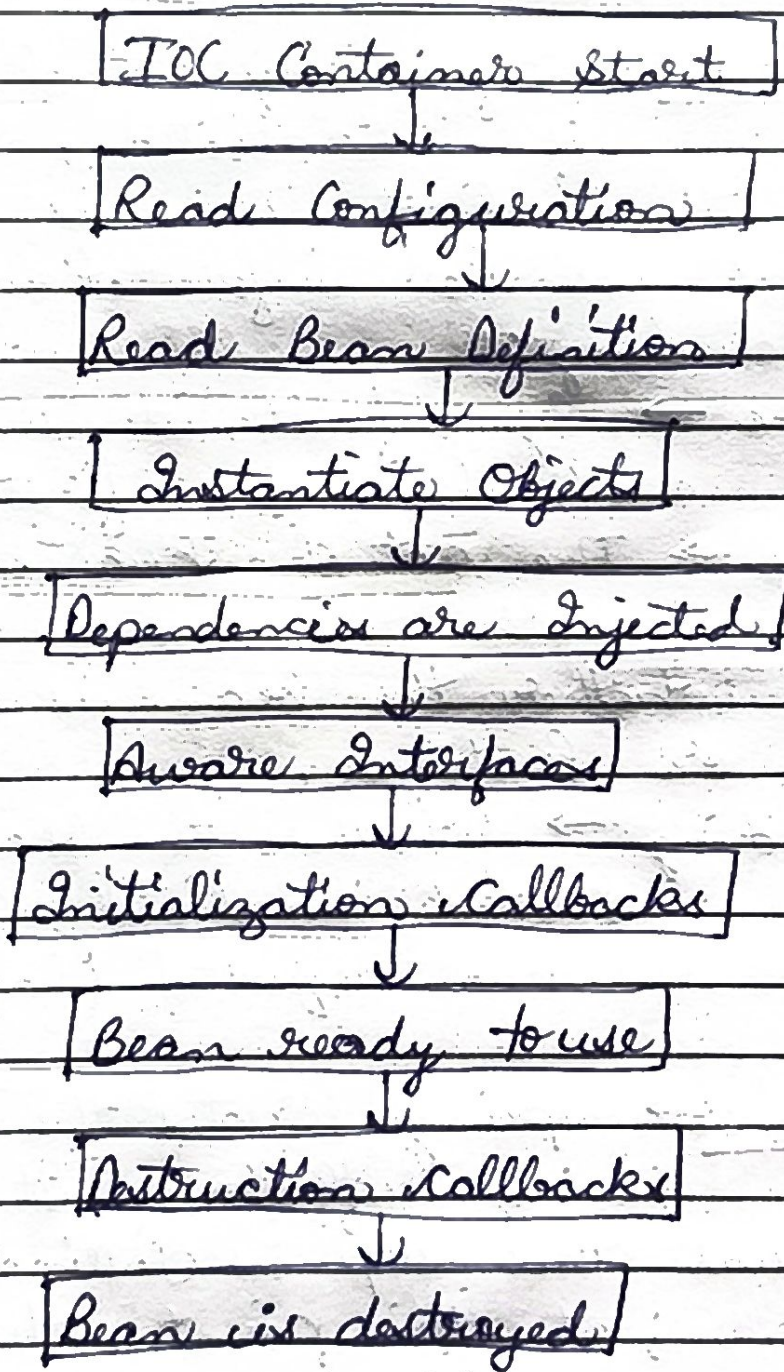


* Spring Bean Lifecycle :-

In Spring Framework, the IOC Container manages the entire lifecycle of objects, known as 'Beans', from creation to destruction.



1. IOC Container Initialization :- The process starts when IOC container

(especially ApplicationContext) starts up. It acts as brain of Spring framework, responsible for entire lifecycle.

2. Reading Configuration :- The container looks for configuration files (like @ Configuration classes) or component scanning instructions (@ComponentScan) to identify which classes are marked as beans.

3. Bean Definition :- Before any actual Java object is instantiated, Spring creates a 'BeanDefinition' object. This is 'metadata' that stores essential information about the bean, including:

- Bean Name → By default, it is class name with lowercase first letter.
- Class Type → The actual Java class associated with the bean.
- Scope → Whether the bean is Singleton or Prototype.
- Lazy Loading → Whether the bean should be created immediately (eager) or only when requested (lazy).

4. Instantiation → Spring creates the bean instance using reflection. It is important to note that this step is distinct from initialization; at this point, the object exists in memory but it is not yet "ready" for full business logic.

5. Dependency Injection :- Spring handles the wiring of dependencies.

- Constructor → The dependency is injected at the moment of object creation. In this case, instantiation and dependency injection happens as a single atomic process.
- Setter/Field → The object is instantiated first, and then dependencies are "injected" or "autowired" into the bean fields or setter methods afterward.

6. Aware Interface :- Before initialization, Spring checks if the bean implements any 'Aware' interfaces (like `BeanNameAware` or `ApplicationContextAware`). These allow the bean to "become aware" of its own name or the Spring container environment by providing specific callback methods (e.g. `setBeanName()`). These are rarely used for business logic but are helpful for infrastructure-level tasks like "logging".

7. Initialization Callbacks :- After dependencies are injected and bean is "aware" of its environment, the bean enters "Initialization Phase". This is the designated stage to perform setup tasks (like clearing caches, loading expensive resources, or validating configurations) before the bean is made available for use.

Spring offers 3 ways:

- InitializingBean Interface → A marker interface where you implement the `afterPropertiesSet()` method. While effective, it tightly couples your code to Spring framework.
- Custom init-method → You can define a plain method in your class (e.g. `public void start()`) and register it in your configuration (using `@Bean (initMethod = "start")`). This keeps your business logic decoupled from Spring.
- @PostConstruct → The modern, preferred approach. By annotating a method with `@PostConstruct`, Spring will automatically invoke it after the bean's dependencies are injected. It is clean, readable and highly recommended.

*Why not use constructor for this?

You might wonder why we don't just put this logic in constructors. The main reason is that dependencies are not fully injected when the constructor runs. If your setup logic requires access to an injected

dependency (e.g. PaymentService), the constructor will fail because that dependency hasn't been linked yet. @PostConstruct ensures all dependencies are ready before your code executes.

8. Ready to use :- The bean is now fully initialized and ready to be used by the application to perform its business logic.

9. Destruction Callbacks :- Just as Spring provides a way to initialize beans, it also provides destruction callbacks to perform cleanup tasks (like closing database connections or clearing resources) before a bean is removed from the IOC Containers.

Similarly, there are 3 ways:

- DisposableBean Interface → You implement this interface and override the destroy() method. Like InitializationBean, this approach couples your class to the Spring framework.
- Custom destroy method → You define a custom cleanup method in your class (e.g. public void cleanup()) and specify it in your configuration using

@Bean(destroyMethod = "cleanup").

- @PreDestroy → The recommended, modern approach. By annotating a method with @PreDestroy, Spring will automatically invoke it right before the bean is destroyed.

10. Destroyed & The bean is destroyed from IOC Containers and resources are freed.

* Prototype Scope & Spring does not manage the full lifecycle of prototype-scoped beans (it creates them and hands them over to developer), it does not track them for destruction.

IOC will not trigger destruction callbacks for prototype. You must manually manage their cleanup, especially if they hold expensive resources.

If not, this could lead to 'memory leaks'.